UNIVERSITY OF DEBRECEN

FACULTY OF INFORMATICS

DEPARTMENT OF COMPUTER SCIENCE

CRYPTANALYSIS OF CRYPTOGRAPHIC ALGORITHMS IN NIGERIAN BANKING SECURITY: JWT, HMAC, RSA256, AES, HOTP, TOTP AND BCRYPT.

SUPERVISOR: BY:

DR. HANNUSCH CAROLIN OGBONNA DIVINE

DEBRECEN 2025

I consent to its submission for review Signature: _____

TABLE OF CONTENTS

CHAPTER 1: 4

INTRODUCTION 4

LITERATURE REVIEW: 5

CHAPTER 2: 9

DISCUSSION 9

BCRYPT 10

PEFORMANCE METRICS: 10

COMPARSIM OVERVIEW 13

JWT 14

PEFROMANCE METRICS: 15

Strength: 15

Speed: 17

Ease of Implementation: 18

Memory space: 19

Real -World's Usage and Adoption: 20

COMPARSIM OVERVIEW 21

OTP 23

PERFORMANCE METRICS: 23

COMPARISM REVIEW 25

CHAPTER 3: 27

GAPS IDENTIFIED WITH POTENTIAL SOLUTIONS 27

ARGON 2 27

JWT + ENCRYPTION (AES-GCM) 28

TOTP 28

CHAPTER 4: 29

SUMMARY 29

CHAPTER 5: 31
BIBLOGRAPHY 31
CHAPTER 6 41
APPENDICIES 41
CHAPTER 7 43
DESCRIPTION OF MY WORK. 43
CHAPTER 8 45
ACKNOWLEDGEMENT 45

CHAPTER 1:

INTRODUCTION

As far as the 90's, the banking processes were difficult, tedious and much more cumbersome, making it almost difficult to access banks at the customer's comfort since physical banks will have long waiting queues. However, when online banking came to light, financial transactions and processes became more convenient and accessible. Despite the positive changes, the high rate of digitalization of banking services also significantly led to the increase of cyber exploitation of citizens with little to no awareness of cybersecurity measures over the years. As crimes have continued to evolve, discussions around the adoption of advanced security technologies are highly relevant thus the exploration of this thesis research. There are several methods used in cryptanalysis, but I will be focusing on performance and security comparison. It is necessary to conduct such analysis as cyber threats increases on a yearly basics in Nigeria.

In this cryptanalysis, we will analyse several cryptographic algorithms which includes JWT with different algorithms like (HMAC, RSA256, and AES), OTP/TOTP and BCrypt for password hashing. The results are evaluated based on their respective security strengths, rate of speed, ease of implementation, memory space they occupy and real – world usage. Findings indicate that RSA and AES are strong in encryption, HMAC needs a hashing algorithm to optimize security, and BCrypt advances the security of password management with its salting and hashing benefits. This result suggests the need for layered approach in combination with JWT for secure authentication, AES for encryption, BCrypt for password encoding and TOTP for added security. With this approach, Nigerian banks can provide robust security in financial processes.

This highlights the need for thorough research on the existing security measures used in safeguarding money and assets in banks.

LITERATURE REVIEW:

Online banking has been since the 1980's in Nigeria and it has evolved since then. (Agbada A.O, 2008). As the evolution of technology emerged, banks both locally and globally have faced thousands of cyber hazards and lost significant amounts of money. It is crucial and essential to safeguard the money from these traitors. This emphasizes the need for cyber security and modern cryptographic measures to help secure funds, sensitive data and renew the trust the masses had for banking intuitions.

Modern cryptography is the backbone for security in various institutions, especially ones related to finances. It entails advanced methods of securing sensitive data of online transactions, electronic payments and money transfers. It generally involves using complex mathematical algorithms to secure these data. There are various techniques in cryptography which includes Encryption, Hashing, Digital Signatures, Symmetric and Asymmetric Key Cryptography.

For this literature review we will explicitly discuss the evolution of cryptography, the common cyber-crime in the banking sector and logically identify the gaps we encountered. As listed before cryptography involves various techniques, but we will focus on AES, RSA and their respective impact in securing bank processing. These crimes include phishing – a form of social engineering that targets vulnerable individuals luring them to disclose their sensitive data. It is no news that Nigerians are well known for cyber fraud popularly known as "The Nigerian Prince Scam" goes about disguising as wealthy individuals who suffers from terminal illness and ask the victim to help them transfer a huge sum of money in exchange for a reward (Lazaricheva, 2025). These victims fall prey and lose their hard-earned money in the process. Another crime is bank fraud which can be in the form of withdrawing funds to gift cards, registration of new accounts with false identities of senior citizens and even minors (Presse, n.d.). Vishing is another form of phishing but this time, scammers use phone, or voice calls instead of tricking gullible individuals through texts. In recent times this has been rampant where hackers will impersonate bank officials or even close friends and relatives asking for the OTP (One Time Password) sent to the victim phone through the email address. (BusinessDay, 2024).

As technology evolved, so did cryptography over the years. It was first founded in 1900 BC then early transposition cipher by Ancient Spartans where they used them to order letters in theory military communications. They are layers of cryptography which are Ancient, Medieval and Modern cryptography. As modern cryptography is our core focus, we will dive into it more. The first ever used cryptanalysis was in World War I in 1913 and it then gradually evolved in World War II to crack German Enigma cryptosystem. (*Schneider,* 2024). The first certified cryptosystem was the Data Encryption Standard (DES) by IBM. There was a need for a stronger key length when DES was implemented and made the application less secure. Due to this effect, algorithms which can compact this were introduced like: Advanced Encryption Standard (AES), Rivest Shamir Adleman (RSA). (*Schneider,* 2024).

Here I would explain how common threats attacks exploit weaknesses in authentication, encryption or data storage. Phishing targets the authentication process by fraudsters disguising themselves as a trusted entity, duping victims into revealing sensitive information like OTP codes and passwords. There are various techniques for phishing which includes: email phishing – Attackers sending out fraudulent messages prompting users to input their credentials or luring them into thinking they can make a significant amount of money. This is designed in such a way that they appear real. This isn't just affected by Nigerians, but the world at large. Spear phishing is designed for a particular person. The attackers had already researched the victims' identities and spots weaknesses they could prey on, like posing as the director of the company they work with. A popular bank in Belgium, Belgian bank Crelan were victims due to this effect as far back as 2016, with a loss of over 70 million euros. (Zorz, 2016). In this case, a compromise was sent to the CEO's email account, urging them to make a payment to a bank account owned by the fraudsters. This emphasizes the need to advance cryptography and cybersecurity measures as the fraudsters are getting smarter with their tactics. The weakness lies in users trusting the phished emails without proper verification and leading to exploitation of the victims' lack of caution.

One of the major cyber heists of all time is the Bangladesh Bank Heist in 2016. (Balu, n.d). The attack was targeted to the Central Bank of Bangladesh. Malware was used to

compromise the bank's network and executed a sum of 35 fraudulent fund transfers. This scenario is typical malware and phishing event. A total sum of 81 million dollars was wired to the Federal Reserve Bank of New York. Spear phishing tactic was used as they researched and found a way to access the bank's network. Weaknesses in this case were exploited in the data storage and encryption protocols. The encryption protocols used by the bank were weak, allowing the attackers to bypass the security making the SWIFT gateway manipulated to initiate transfers. It was reviewed by investigators that it was caused due to the weak encryption and poorly stored credentials that the attack took place.

Now we can dive into the cryptographic techniques listed and how they impact the banking system. The Advanced Encryption Standard (AES) is one of the highly trusted and secured encryption algorithms that replaced the Data Encryption Standard (DES) and triple DES. This algorithm is a block cipher with key sizes ranging from 128 to 256 bits. The AES is used for its speed and security. It has 4 main operations which includes the sub-Bytes (Substitution), shift Row (Permutation), Mix column (Diffusion) and Add Round Key. (*Zenarmor*, 2021). Each stage of the process strengthens the encryption. The 128 bits have a sum of 10 rounds and the highest round is about 14. It has widely been used and even standardized by the National Institute of Standards and Technology (*Zenarmor*, 2021). Studies by (Jadhao & Kumbhalkar, 2016) and (Onesafe, 2024.) emphasize the importance of securing banking processes and transaction data using AES encryption from potential attacks and even quantum threat to cryptography. However, some research has found weaknesses with this encryption, including that they are highly resistant to attacks like brute force, Key management is still a big issue which could potentially weaken the AES security, and they are still vulnerable to advancement in quantum computation where AES-128 security level can be reduced to that of 64 with Grover's algorithm. (Farik, 2017). This study ((Flynn & Chen, 2015) also shows that AES-256 decryption can lead to leaks in power consumption patterns that attackers can exploit to recover encryption keys and security in an embedded device is insufficient.

Another cryptographic algorithm we will talk about is Rivest – Shamir – Adleman (RSA) which uses public key cryptosystem – this asymmetric encryption involves two keys which are public and private. The data encrypted with the private key can only be

decrypted with the accurate and corresponding public key. It relies on computational difficulty of factoring large integers. The common cases and applications are security of communications, digital certificates, secure key exchange and financial transactions. The generation of the public and private key is the most complex part of RSA as it deals with a lot of calculations. (*Cobb, 2021*.). Studies, however, have shown several vulnerabilities in this algorithm. It has been proven that quantum algorithms can quickly factor large numbers, making RSA less secure. Also, the security is directly proportional to its large, difficult-to factor prime numbers used in the generation of the key, which can weaken the private key if poorly generated due to its randomness. The factoring process of the product of two large prime numbers is more difficult to calculate when the length of the key is higher. (*Cobb, 2021*).

After extensive research there is a lack of comprehensive studies on cryptographic algorithms which emphasizes the need for proper documentation. As cybercrime continues to evolve, understanding and proper implementation of cryptographic techniques is highly crucial to ensuring a reliable financial system.

CHAPTER 2:

DISCUSSION

The method follows a theoretical approach with the revision and in-depth analysis of existing literature and papers. We will focus on the core algorithms used which includes - JSON Web Tokens, RSA256, AES, HMAC, TOTP/OTP and BCrypt password hashing in banking security. The results are evaluated based on their respective security strengths, rate of speed, ease of implementation, memory space they occupy and real – world usage. It also entails some limitations identifies during analysis. For each algorithm the following method will be used:

Security Strength: How these algorithms can resist attacks and threats and vulnerabilities like Bruce force, token theft, key management issues etc.

Rate of Speed (Execution time): How fast and accurate can they process data in reference to encryption/decryption, token generation, hashing, authentication.

Ease of Implementation: How complex is it to set up the algorithm. Difficulty level to code and time to implement.

Memory Usage: How much memory is consumed by each algorithm while executing their respective process?

Real-World's Usage and Adoption: How practical and efficient the algorithm is. How well do they perform in real-world environments like banks and databases.

The above metrics were used to measure performance because of the sensitivity of the data banks handle. An analysis of the levels of security can minimize weaker authentication which could lead to greater damage. Speed and accuracy are the core of financial processes to avoid long waiting lines for customers. We don't want to cause any inconveniences to the customer and increase the server load. This great right balance between security and performance is very important and finally, Memory space is taken into consideration because of the millions of transactions that occur daily in local banks, the need to examine the storage is needed. Ease of implementation and real – world applicability was necessary as it gives the cryptanalysis more in-depth making it relevant in recent times. It also

ensures that the algorithm is in accordance with regulations, tamper–proof and can be easily integrated without compromising the security of the system.

By proper analysis of these three metrices, this cryptanalysis will help Nigerian banks choose the best security methods for their respective needs.

BCRYPT

Simple hashing algorithms aren't enough as they have lots of vulnerabilities and weaknesses (Secure Flag, n.d). As technology advances, so is the need to advance password hashing. Hashing in simpler terms means the process of transforming or changing a key into another value for the purpose of security. It is like standard encryption but differs because unlike encryption which is two –way, hashing is often one way. In this section, I will compare the two most common password hashing algorithms. They are BCrypt and Argon2. BCrypt uses a Blowfish cipher – This is a fast block cipher used for encrypting data. This type of hashing was introduced in 1999 but is still relevant to this day. It assists in reducing Bruce force or rainbow table attacks, increases security by adding a work factor. (Kinde, 2024). On the other hand, Argon2 is a newer algorithm, founded in 2001. It won the Password Hashing Competition (PHC). It has three versions: Argon 2d, Argon 2i and Argon 2id. The versions have high resistance against GPU cracking and side-channel attacks. (Argon2, 2024). As we dive into this analysis, we will see which is better to meet finical needs as many security experts recommend BCrypt why many systems still use BCrypt.

PEFORMANCE METRICS:

Here we will analysis critically to evaluate the strengths, weakness and the use cares.

Security: From the findings, BCrypt uses Blowfish cipher – symmetric-key clock cipher used for encryption and password hashing. BCrypt is secure to use due to its hashing process.

Random pieces of data are added to increase its security and make it hard for various attacks to exploit like brute force. Aside from hashing, BCrypt also salt the result which enhances the passwords more. (Kelseyk, 2023). However, this comes with several use cases that exploit the vulnerabilities of this system like the Okta Vulnerability in 2024, this

vulnerability was revealed where the password was not concatenated properly even after it has been hashed with BCrypt. (*Bureau,2024*). Also, according to analysis, (*Strapi,2023*.) BCrypt cost factor is 10 which is relatively low which makes it susceptible to brute force attacks and increasing the cost factor was advised.

Argon2 has a high resistance to ASIC based and GPU- based attacks. It can even further withstand brute force, dictionary and rainbow attacks. Another major advantage is its ability is the resulting string has various features like variant, version, memory cost, time cost, parallelism factor, salt and resulting hash, which provides added layer of security. (*OnlineHashCrack* n.d.). The award won by Argon2 emphasizes the effectiveness of security.

Speed: BCrypt is generally slow hashing, since it does the rounds several times before a hash is done. It is also expensive for this. It takes about 250ms to 500ms to compute on the server hardware and depending on the high-cost factor, several rounds are needed as the factor increases. (*Kim*, 2024). Due to this effect, it functions better in slower operations and is CPU intensive and this algorithm scarifies speed for advanced security.

The Argon2i and 2d have a high CPU cycles per byte. Meaning, it fills memory much faster, it often CPU efficient and can take approximately 50ms to 200ms to be configured. It is used in banking, cryptocurrency and applications that need memory-hardness. (Biryukov et al., 2015)

Ease of Implementation: BCrypt is widely known for its simplicity and easy integration. It has so many pre-built libraries in popular programming languages like Python, Java, and Nodejs. I will attach some simple implementations below. This library enables developers to handle the complexities of salting and hashing seamlessly. The BCrypt algorithm creates a space that balances security for authentication and implementation. For example, here is a simple implementation to has and verify password with Nodejs

(https://www.freecodecamp.org/news/how-to-hash-passwords-with-bcrypt-in-nodejs/). The process entails – creating a directory and file, installing the dependencies, salt and hash the password and finally verifying the password.

Argon2 has a major strength of configuration of the algorithm, making it very flexible as you can set the number of iterations you need, memory and parallelism cost. For JavaSpringBoot framework, it implements another library called Bouncy Castle. Here is a comprehensive link for the implementation. (https://www.baeldung.com/java-argon2hashing). Argon with its flexibility ability is slightly more complex than BCrypt because of its extra configuration of the three parameters needed to be set by the developers.

Memory: BCrypt is not a key derivation function (KDF) unlike argon2. It doesn't have memory-hardness. It is designed to be slow to reduce brute-force attacks It memory is highly dependent on the number of iteration rounds. The cost factor also affects memory consumption, as the higher cost factor, the more memory is used. It is CPU bound. It has been stated that argon2 is a memory–hard function and has the highest memory-filling rate. It means the memory usage is highly configurable. Its memory size is about 8*p to 2^ (32)-1 kibibytes. By increasing the memory size, it becomes harder to perform attacks like brute force and parallelism. If a system has limited memory, performance might be affected. (Biryukov et al., 2021)

Real -World's Usage and Adoption: BCrypt is widely adopted in different institutions.

Okta is an American identity and access management company who render IAM services to financial institutions. It provides various services like Single Sign-On (SISION), Multi authentication security *(Okta | the Identity Standard, n.d.)*

It is specifically stated here (*Bureau, 2024*.) that they implement BCrypt algorithm making it very useful in recent times. They are used for ensuring security for user authentication.

A company founded in Germany called Ory Kratos specializing in identity and access management (IAM), open-source access control user management software has indicated

that they use Argon2. It is a very flexible and secure algorithm. In this paper it emphasizes that the algorithm isn't only secure but recommended by OWASP. (Neu, n.d.).

COMPARSIM OVERVIEW

Metrics	BCrypt	Argon2
Security	Reduce to brute force attacks, slower but often sacrifices speed for advanced security	Withstand several attacks like brute force, rainbow dictionary, ASIC based and GPU-based attacks
Speed	Slower due to its hashing process and its CPU intensive	More CPU efficient and fills memory faster. Used for applications with memory hardness.
Ease of implementation	Easy to implement and integrate because of its vast properties like simplicity and pre-built libraries.	Flexible to use because of the ability to set the number of iterations you need, memory and parallelism cost.
Memory	Designed to slow down brute force attacks and doesn't require memory hardness. Cost factor can affect the memory.	Memory hardness and highest memory filling rate. It is also highly configurable.
Real-world use	Used in companies like Okta for multi – authentication and SISO security.	Used by companies like Ory Kratos for its flexibility and security.
Adoption in Nigerian Banks	Used worldwide but not explicitly adopted in Nigeria	Used worldwide but not explicitly adopted in Nigeria.

This compares BCrypt and Argon2 for banking security, both have their Pros and Cons as BCrypt can be easier to implement but hashes slowly and Argon2 is CPU efficient but can be memory intensive for limited memory systems. One of the most common attacks in Nigeria is brute force, from this study

(Oladapo,2024), it is stated that the National Information Technology Development Agency (NITDA) has emphasized the need for better security, in our case Argon2 can be used because of its high resistance to brute force attacks even though BCrypt has some ability to reduce the attack too. Argon2's flexibility enhances the security without compromising the security for the Nigerian banks unlike BCrypt that can only adjust the work factor, and the resistance can be limited due to memory hardness. The development of BCrypt was way before Argon2, making it less effective in this day as cybercrime keeps evolving. From the explanation above, with the awards won by Argon2's algorithm, it emphasizes the flexibility, memory efficiency and resistance to many more attacks not just brute force which will greatly improve the security in Nigerian banks. As Nigerian banks incorporate Argon2 to the banking security, it can enhance the defence against future threats and emerging cyber risks.

JWT

For this section, I will dive into different scenarios JWT can be paired with in respect to banking security. We will compare several of them adequately. First, we will look at JWT with its standard HMAC, its performance metrics in respect to the categories listed above. Then we compare it alongside RSA256, EdDSA, and finally JWT+ Encryption (AES). We outline the trade-offs between each performance and which approach is best suited for Nigerian banking security needs.

For easy breakdown, here is the overview of the comparison:

- JWT with HMAC (standard approach)
- JWT with RSA256
- JWT with EdDSA
- JWT with Encryption (AES)

JSON web token (JWT) is an open standard RFC 7519 – this is the standardized structure the JWT must follow. It comprises sections of structure, claims types and how signing and

verification can be achieved. It is used for securing information, authentication and authorization that is being transmitted as a base64 encoded JSON object. The claim types are used as the payload of the web signature which enables the digital signed with a Message Authentication Code (MAC).

A claim type is often in the form of string that acts as the object. It can be represented as a name or value (Jones et al., 2015). The JWT which has been signed already is known as JWS and typically contains the header, payload and signature. I will briefly explain each one of them

Header: It often contains the metadata about JWT where you can find the key identifier and the algorithm that is used to sign it.

Payload: This is also called the body of the token. This contains where JWT was created to transport such as the roles or the authorization info.

Signature: It is by far the most critical part of the JWT because it guarantees the integrity of both the payload and header. (*Moore, n.d*).

In banking JWT is widely used for its stateless authentication and authorization purposes. It is self-contained which means it holds all necessary information within the token itself which eliminates session hijacking. It is also ideal for distrusted systems like microservices which prevents the need for centralized session management making it easier to scale horizontally.

PEFROMANCE METRICS:

This section evaluates the efficiency of JWT with different algorithms are as follows:

Strength:

JWT with HMAC: The strength of the HMAC depends on the size of secret key and the specific hash function used. From this research paper, it is stated that HMAC uses MD5 or SHA as its hash functions. It was also stated that HMAC needs to be tied with a hash algorithm to obtain maximum security. (Bellare et al., 1996). Here, (Bejamas, 2024) we can see recently there are several vulnerabilities with HMAC where attackers can access the public key if there is a problem with the processing, if the server expects an asymmetric

algorithm but gets the symmetric one. It can cause mismatch between the algorithms and since HMAC relies on a shared key for both verifying and signing it will lead to attacks like brute force.

JWT with RSA256*:* The combination of RSA256 includes using RSA for encryption and SHA 256 for hashing. This form is asymmetric encryption where two keys are used in pairs (public and private keys) for encryption and decryption. (*GeeksForGeeks, 2025*). From this paper (Andres, 2018), It is indicated that RSA256 the recommended key size is 2048.

JWT with EdDSA: Just as the default algorithm used for JWT is RFC 7519, Edwards-Curve Digital Signature Algorithm (EdDSA) uses RFC 8032 instead. This algorithm is a variant of Schnorr's signature system in conjunction with Edwards's curves. There are different security levels of EdDSA, like ED25519 and ED448 having 128 and 224 bits consecutively. One of the key strengths of this algorithm is that it has a resistance to side-channel attacks by ensuring the implementation when preforming signatures is executed correctly and accurately in the same sequence of instructions and memory, for any value of the private key. (Josefsson & Ilari Liusvaara, 2020).

JWT with AES: Advanced Encryption Standard (AES) is a symmetric block cipher for encryption. There are different modes in AES which are as follows: Electronic Code Book (ECB), Cipher Block Chaining (CBC), Cipher Feedback (CFB), Output Feedback (OFB), Counter (CTR), Galois Counter Mode (GCM) and XEX – based Tweaked Codebook Mode (*PGzlan*, 2023). From this research (Kariyawasam, 2021), it has been proven that the GCM mode- which consists of combination of Authentication and Counter mode is faster and more secure for implementation than the other modes. JWT doesn't offer this property, it is wise to add some layer of security by encryption. It has various blocks and the key length ranges from 128 to 256 depending on the type of AES you are using. *(Bernstein et al., 2024.* JWT in connection with RSA and AES serves different purposes, AES encryption is used to secure the refresh token, and RSA ensures that only the correct server can encrypt the AES key. This combination minimizes the risk of token theft and enhances the speed and efficiency of handling larger pieces of data. (Lê Hoàng Tâm, 2024)

Speed:

JWT with HMAC: This is the most common approach; it is lightweight and faster than the traditional session-based authentications because of its stateless nature. This is more scalable and efficient than session-based authentications. HMAC algorithms shares only one key, it is symmetric in nature, thereby making implementation a lot easier. (Bodewes 2019). However, according to this paper (Soni, 2024), there are several algorithms which can be faster than JWT algorithms like Ed25519 and Edd48 versions of elliptic curve signature algorithms causing a gap.

JWT with RS256: The algorithm in RS256 is more CPU intensive which occupies the time used for verification. It was exposed that RS2S56 performs about 13,781 verifications er second. (*How to Improve JWT Performance in Node.js* | Nearform, 2020). It is also noted that RS256 isn't safe for encryption again and it was advised to use better options. (Brady, 2020).

JWT with EdDSA: EdDSA is very fast in nature, particularly Ed25519 for generating signatures. As the name implies, it used elliptic curve structure which has a few steps in comparison to slower algorithms like RSA 2K. This algorithm is able to sign over 30,755 signatures per second. However, for verification, RSA is much better than EdDSA with over 22,000 more verifications per second. (Buchanaa, 2023).

JWT with AES: To determine the highest performance of the JWT with AES, we will use various benchmarks like Vanilla Java, Bouncy Castle, Apache commons crypto and Clojure buddy library. It was showed that Vanilla java was the fastest with over 40722.2671 to 76144.372 where AES- GCM was the fastest algorithm used and with the lowest overhead while Bounty castle was a bit slower and Clojure buddy library having the worst and closer performance. CBC-HMAC was also slower than GCM. (Vuuren, 2019).

Ease of Implementation:

JWT with HMAC: This is often used in Nimbus JOSE+JWT library. HMAC has various lengths like H256, HS384 and HS512. All require different number of secret length bits. Here is the link attached to see the implementation in Java. (https://connect2id.com/products/nimbus-jose-jwt/examples/jwt-with-hmac). From this paper (*Jwt Security Best Practices | Curity*, n.d), it is stated JWT with HMAC256 is the best when it comes with symmetric signing, but it is more difficult to guide the secret and assure safety of the key. Hence, it is advisable to use ephemeral secrets to increase security.

JWT with RSA256: RS256 requires 2 pairs of keys as stated earlier. The private key is used for signing of the tokens and the public key is used for verification. It is easy to implement and supported by various programming languages. Here is an example of how it is used in Java. (https://ajinkyakarode24.medium.com/jwt-authentication-using-rs256-algorithm-injavaa3dbb9cacb78). With the security of RSA256 comes complexity in the management of keys. The private key must be stored securely to prevent possibilities of exploitation. (Adeyeye, 2024).

JWT with EdDSA: EdDSA uses both private and public key, where the private key is an integer, and the public key is a point (x, y) on the curve. It differs from ESCDA even though they both operate on elliptic curves. The Y coordinate of the curve point is encoded, and a bit is reserved for the x-coordinate to be stored. A new key is later introduced called Octet Key pair to be able to encode the key as a JSON Web Key (JWK). EdDSA is deterministic and does not depend on random number generator making it less vulnerable to attacks. Here is a simple implementation of the algorithm. (https://curity.io/resources/learn/jwt-signatures/)

JWT with AES*:* JWT with encryption enables the JWT recipient security from unauthorize use and can't be easily manipulated. This builds on two properties of core security – confidentiality and integrity. AES GCM can be used with different keys ranging from 126, 192 and 256- bit key. Key management is critical for this algorithm, which is why we have over 17 standard Key Management Algorithms. The AES Key Encryption includes:

A128KW, A192KW, A256KW, A128GCMKW, A192GCMKW and A256GCMKW. New secure - random key is generated using the AES Wrap algorithm which allows the JWT with encryption (JWE) with a random short-lived key. (*Ihazlewood, 2022*). Here is the comprehensive overview implementation of the algorithm.
(https://github.com/jwtk/jjwt?tab=readme-ov-file)

Memory space:

JWT with HMAC: This algorithm uses the HS256 or the HS512. The HS512 has a longer byte of about 64 bytes making it longer in length compared with HS256. Longer keys enhance the efficiency of security but often take up more space for storage. JWT can be stored in different ways which includes Local, Cookies, Session and In-memory storage. All have Pros and cons, but all are vulnerable to XSS attacks – known as cross-site scripting, which involves manipulation of vulnerable websites making them insufficient except additional security is added. (Bejamas, 2024).

JWT with RSA256: This algorithm which is about 2048 bits or higher has been confirmed that it consumes memory due to its large key size and complex computations. For instance, discussion on GitHub platform where developers highlight the memory considerations when implementing JWT with public keys. (Envoyproxy, 2021).

JWT with EdDSA: This has different key generated types which are Ed25519 having 32 bytes and Ed448, 72 bytes. This small number of public and private keys provides a maximum level of security. Unlike ECDSA, EdDSA is simpler to implement and is slightly faster than ECDSA. (*EdDSA and Ed25519 - Practical Cryptography for Developers*, 2022)

JWT with AES: It is essential to understand and consider how each mode operates and the resources that are required by them. This study (Wang, 2019) was about AES- 128 with different modes such as ECB (Electronic Code Book), CBC (Cipher Block Chaining), CFB (Cipher Feedback), OFB (Output Feedback) and CTR (Counter). It was concluded that ECB is the fastest mode but not recommended. It was also seen that as the key length increases, the speed of the encryption decreases. It was checked with various bytes from

16 to 8192. In comparison with CBC and Galois/Counter Mode (GCM), CBC lacks built in authentication, isn't parallelizable and can only be used for local files (The Quantum Physicist, 2018). The length of the AES-CBC is often calculated as L = 16* (floor (M/16) +2) where M denotes the number of octets in the plaintext, floor rounds the argument down to the nearest integer (McGrew, 2009). Unlike CBC that requires padding, GCM doesn't require padding as the length of the ciphertext is equal to the plaintext and a 16-byte authentication tag is often used for verification of its integrity. (*AES GCM Stream Spec - Apache Iceberg™*, n.d.)

)

Real -World's Usage and Adoption:

JWT with HMAC: In Ngrok, webhook- framework used to send updates to other applications as events happen which are used by different companies like Stripe for recurring payment and Twilio for SMS message. The HMAC SHA 256 is used in this case. Ngrok is a company that provides secure tunnels to expose their local serves to the internet. (*Frederico, 2024*).

JWT with RSA256: Integrating JWT with RS256 is a common practice, particularly Azure Active Directory (Azure AD). This is a Microsoft documentation that tokens are signed using RSA256. (Cil Werner, 2023). Azure AD ensures that the organization is safe as they provide cloud identity and access management solutions. It has different capabilities like password less and multifactor authentication, identity protection, privileged identity management etc.
(*Microsoft Entra ID (Formerly Azure Active Directory) | Microsoft Security*, n.d.)

JWT with EdDSA: A company called Curity which deals with Identity Management for APIs and mobile banks uses one of their products named Curity Identity Server which is designed to implement modern security protocols considered elliptic curve-based algorithms to be more secure particularly EdDSA. (*JWT Token Security Best Practices | Curity*, n.d.) This demonstrates the significant role algorithms like EdDSA have in securing digital transactions and ensuring user authentication.

JWT with AES: Companies like VISA – world leader in digital payments offering several security features like zero liability, network security and card technology (*Visa*, n.d.) has an outline for AES-GCM –256KW algorithm for the JWT. It was also stated that they use Content Encryption Key (CEK) which has 256 bits and compact serialization style is used where element is separated by "." (*Jwe-Jws*, n.d.).

COMPARSIM OVERVIEW

Metrics	JWT +HMAC	JWT +RSA256	JWT +EdDSA	JWT + AES
Security Strength	Strength depends on the size of key and hash function used. Vulnerable to brute force.	Strong but slower than EdDSA but it has also been proven otherwise.	Strong and comes with different perks like it's resistant to side channel attacks.	Strong (combines Authentication and Counter mode)
Speed	Faster than the traditional OTP	CPU intensive with about 13,781	Fast but has some pitfalls as it is slower for	AES- GCM was the fastest algorithm used
	making it more scalable.	verifications per seconds	verification. It can sign over 30,755 signatures per second	and with the lowest overhead
Ease of implementation	Widely supported but best when you use symmetric signing.	Simple to implement but comes with the complexity of key pair management.	Complex as it needs proper key encoding for functionality.	Requires key management for easier access.

Memory space	Needs longer keys to function and increase its security capacity	Requires more memory space of about 2048 or more bits to function properly.	Requires less memory space of about 32 and 72 bytes depending on the algorithm used.	High due to authentication tag but does not require padding like AES-CBC
Real world usage	Used globally in companies like Ngrok, Stripe and Twilio.	Used in companies like Azure AD for multifactor authentication, identity protection, privileged identity management	Used in companies like Curity as a security function.	Used in world leading companies like Visa for digital payments.
Adoption in Nigerian banks	Used but moderately for security	Used for secure banking transactions	Will be used for as a future innovation. Still needs room for advancement	Limited case study but can be possible in modern banking APIs

From the detailed analysis above, JWT with EdDSA is the strongest if it is without encryption, but since encryption is needed, there is a need for AES-GCM. AES-GCM will strengthen the security of the system. In this study (BusinessDay, 2024), it was stated that over 1.3 million phishing attempts were declared in Nigeria alone for the first half of the year in a report in 2023 by Check Point Research. However, it requires the complexity that comes with managing keys which might be difficult. The memory space is also high due to the authentication tag. With this defect, we can use the strong signing algorithm – EdDSA and AES-GM for encryption which will help with the key management and memory space. By incorporating this technique in Nigerian banks, attacks like phishing will be minimized since integrity of the token is maintained through the GCM tag and sensitive data can be encrypted to prevent theft.

OTP

One Time Password (OTP) as the name implies, can only be used once. It adds a layer of security because of its changing and dynamic nature. The OTP implementations are based on the standard Open Authentication (OATH). There are two types of this based on event and time synchronous. HOTP is an event-based OTP and TOTP is time based. We will discuss the OTP and the HOTP in detail later. (Markström, 2022).

PERFORMANCE METRICS:

This section evaluates the OTPs:

Strength: HOTP as a standard approach to OTP has an additional hash function of SHA-1. This authentication generates a new passcode when a button is pressed. It is counter value. This is used for systems when time isn't critical. This ensures that codes which are verifiable can expire only after one use. Therefore, making it immune to replay attacks. (*Santhanam,2024*). However, this writer found some weaknesses in the passwords. It is vulnerable to Sim swap attacks where attackers take possession of the victim's number and do some exploitation. It is also easy penetrated through phishing and social engineering attacks. (Keyless, 2024)

TOTP is implemented through different third-party applications like Google Authenticator and Authy. This is valid for a shorter period i.e. 30 seconds which minimizes the window for attacks. However, there are still pitfalls that can cause exploitation if not accurately implemented like large number of validation window, secret length etc. (Doan, 2024).

Speed: HOTP execution time is slower than that of TOTP because of the counters between the client and user side which often requires resynchronizations. TOTP eradicates the need for synchronization and is therefore more secure. It also states that TOTP requires authentication server for it to be able to handle potential time drift. (Lumburovska et al., n.d.)

Ease of Implementation: The HOTP is defined in RFC 4226- The RFC also known as Request for Comments 4226 helps in the generation of OTP based on a counter and a shared key. (M'Raihi et al., 2020). This is a comprehensive implementation of this using JavaScript and SHA1. https://github.com/adulau/hotp-js.

TOTP is quite like HOTP, but it generates a code that has a time window of 30 seconds. Before the time is set, the share key generated is shared between the user and the authentication devices, it then uses cryptographic algorithm like SHA1 or generate the TOTP. For diversity, we will see the simple implementation on Golang. It is also important to know that the TOTP doesn't require internet connection making the experience seamless and strikes a balance between security and convenience. Here is an overview of the implementation. https://permify.co/post/two-factor-authentication-2fa-totp-golang/

Memory space: The HOTP requires two inputs from the server and the client side. The result after being hashed is about 160-bit long string but later shortened to about 6 or 8 digits. For the server side, the counter is incremented only if the password is validated and can lead to synchronization issues if the counter is not updated accurately. (Oliynyk, 2020) TOTP is very similar to that of HOTP but instead of a counter, a current timestamp is used on the client side which eradicates the use of counters. However, it needs at least 128 bits generated to resist brute force attacks. The TOTP is defined by REF 6238. The validation therefore doesn't need a counter but based on time intervals. (Arent, 2024).

Real -World's Usage and Adoption: HOTP should be used by different companies and industries. For example, Yubico – a company which supports multiple authentication and zero trust architecture which helps to prevent cyber threats and are often the standards of compliance requirements. (*Yubi.co, 2024*). This company uses HOTP for the YubiKey for security. (*Oath-Hotp*, n.d.). The spunk has also confirmed that there isn't a security issue and has a total of 614 weekly downloads. (*Choose the Best Package - Snyk Open Source Advisor*, 2025). TOTP is used for various services because of its security features. In the financial sector, it is used for online accounts as a second factor authentication option and for secure password managers for similar purposes. (Arent, 2024). However, researchers are seeing weaknesses with the traditional TOTP. Oasis security has demonstrated that if the time window is extended combined with a lack of rate limiting, it

can serve as an entry to attack and even give the attacker about 3 prevent of guessing his way within a single validity window. (*Proser,2025*.).

COMPARISM REVIEW

Metrics	HOTP	TOTP
Security	Less secure due to counter based generation and synchronization	More secure as OTP seems have a shorter time window and can help reduce replay attacks
Speed	Slower due to its counter synchronization	Doesn't need counter synchronization only time based which is faster.
Ease of Implementation	Additional handling is required for counter to be synchronized between client and server	Easier as no need for counter synchronization
Memory Space	Uses as long as 160 bits because it requires storage of the counter value	Often lesser but will need at least 128 bits to be able to resist Bruce force attack.
Real-World Usage	Used in various companies like Yubico but it can be replaced with TOTP.	Used in banking sectors for authentication and transactions and is often more secure
Adoption in Nigerian Banks	Used but its less common due to its defects	It is more commonly used for mobile banking.

HOTP has so many limitations as it heavily relies on counter synchronization that if the counter goes out of sync, authentication can fail. TOTP eliminates the need for this process as it is time based, making it a better solution in Nigerian banks. The Network for the Actualization of Social Growth and Viable Development (NEFGAD) has reported OTP abuse in Netflix Nigeria and United Bank for Africa (UBA). This case was stated as an

illegal withdrawal of funds from Mr. Akingunola′s account a Nigerian citizen. This led to numerous debits just for a single subscription. It was also discovered that OTP was used fraudulently and should only be used for a single transaction instead (Okanlawon, 2021). This emphasizes the flaw in OTP –based authentication used in Nigerian banks and need for better OTP procedures. TOTP offers more advanced security and can reduce the risk of these attacks in Nigerian banks

CHAPTER 3:

GAPS IDENTIFIED WITH POTENTIAL SOLUTIONS

As cybercrimes and technologies are evolving, intruders are beginning to use the weaknesses of these algorithms to bypass the security of the systems. That is why it is essential to identify the gaps in the algorithm in relation to Nigerian banking and propose possible solutions that needs to be implemented. In this section, I will highlight some key defects in the algorithms that were chosen to be a better fit after comparison with some solutions to combat them.

ARGON 2

We have concluded that Argon 2 is a better password hashing algorithm than BCrypt but it comes with its own defects. From this study (*Go-Gitea, 2021*) by tech experts on GitHub. It was stated that using this algorithm comes with great consumption of memory. This might pose a problem because it is expensive and even harder to maintain. This makes scaling for larger companies with massive customers like banks harder to implement such algorithms. The gap identified needs to be solved by reducing the memory usage and cost of the algorithm without compromising security of the sensitive data. One possible solution is cooperation with global companies that use the algorithm already and have specialized software for it. For instance, through partnerships with big financial institution, access to specialized hardware's for these operations can be utilized by Nigerian banks at a much lower cost. Also, smaller countries with higher risks for cybercrimes in Africa can work together and share the implementation and operational costs, this way, this is like killing two birds with one stone which can lead to reduction of the financial strain caused. We have discussed the cost, now let's dive into how we can reduce memory usage. Since security is highly dependent on memory, reduction of memory usage will have an effect on the security standard. We introduce a parallelism approach to the system. This feature will be used in such a way that multiple operations can happen simultaneously and can be adjusted to be memory-intensive depending on the specifications. This way we are distributing the workload across multiple processes so we aren't reducing the memory usage per say but the processing will be more efficient across all resources available.

JWT + ENCRYPTION (AES-GCM)

From the comparison, JWT with AES- GCM was the fastest algorithm used and with the lowest overhead. While this is strong for encryption. There are still vulnerabilities, like this study (RubySec, 2018) where length of authentication tag isn't checked, and attackers can force a valid signature on the average of 128 (max 256). This emphasizes the need of proper and careful implementation with the algorithm to prevent this from happening. There needs to be a series of checks before production to prevent inadequacies. Another defect is libraries like nodejose, jose2go are vulnerable to attacks like the invalid curve attack where the decryption/signature verfication key can be under the intruders control. (*Auth0,2017*). Ultimately, there need to be rigorous validity of the libraries and standardized checks to prevent this flaw. Nigerian banks can incorporate this and be two steps ahead by regular checkups and proper awareness to the developers to prevent such attacks.

TOTP

TOTP is easier than HOTP from the comparison above as there is no need for counter synchronization and requires less memory space making this a better approach. However, it is still vulnerable to several attacks like phishing, key theft, sim swapping making it not efficient either. (*Security, 2025*.).

TOTP, therefore isn't the best algorithm to use now, that's why I am proposing a push button approach, and for instance, for phishing attacks, the push button can reduce the risk by just pushing the button for approval or denial of login attempt. In this way there are no codes to steal. There are several advantages of the push button as this is a step further to increase the standard of security in Nigerian banks. Companies like Authy (*Everybody should 2FA, n.d*) are already implementing this approach but it comes with a price to pay, which is availability of internet connectivity with is minute compared to the increase in security.

This approach in conjunction with the several algorithms above will provide a robust and comprehensive security protocol that will significantly reduce the cybercrime in Nigeria.

CHAPTER 4:
SUMMARY

This chapter will conclude the cryptanalysis by summarizing the key research findings, as well as the value and contribution thereof. It will also review the limitations of the study and propose opportunities for future approach. This analysis raised awareness of the high rate of cyber threats in Nigeria caused by hackers and the need to advance the security measures in the country. With the evolution of technology, banks are encouraged to advance their security measures. It also explored some key cryptographic techniques aimed at security banking processes. It was discovered that many gullible Nigerians are having a hard time trusting the banks with their money. Although new security measures are emerging, money kept in the bank is also at high risk since the rate of attackers keeps increasing. Customers' trust can be lost if the security system does not make diligent changes. The project is an advancement of the Nigerian banking system, yet cybercrime being so dynamic requires greater integration and enhancement with financial bodies for better security. It also validates the need for effective measures and policies to be put in place regarding decision making and actual practices in the banking sector, especially with the high rise of cyber threats.

It was discovered that Argon 2 is a better password hashing algorithm than BCrypt, but it comes with its own side effects like high rate of memory consumption, that is why I proposed a solution of partnerships with big financial institution and the use of parallelism on the system. This approach when implemented correctly can reduce the cost and the memory usage. JWT with AES-GCM was stated to have the lowest overhead between the algorithms listed but it was also exposed to vulnerabilities like lack of careful implementation. Due to this effect, each algorithm should go through a well detailed and necessary standardized check before implementation. Though TOTP adds security compared to HOTP, the push notification button was suggested to reduce attacks the TOTP was vulnerable to like SIM swapping. This approach is seamless and prevents the stealing of OTP codes since no codes are involved. It was also suggested that a combination of all the modern cryptographic algorithm listed will further improve the overall security in Nigerian banks.

While the study is detailed and provide practical solutions, there are several limitations to consider. Banking security entails several physical and digital security like network security, data protection, transaction monitoring, fraud detection but this study only emphasized on Encryption and Authentication. Though, several modern cryptographic algorithms were mentioned, other modern cryptographic algorithms were not stated such as Quantum-resistant systems. Limited test environment and access to banking systems posed a challenge also as it was difficult to obtain real – world datasets for testing. However, these limitations do not diminish the quality of the research findings.

Despite issues listed above, data and the nature of cyber threats that keeps changing, the research points out the importance of improving cyber security strategies. Nevertheless, the prototype is subject to further advancement like proper exploration of post- quantum cryptography, artificial Intelligence and machine learning can be incorporated for real time fraud detection as technology keeps increasing likewise cyber attackers. To maintain clients' trust and confidence in banking services, financial institutions must be proactive and agile in confronting obstacles to stay ahead of escalating cyber dangers.

In conclusion, the study emphasizes the need for and importance of multi-layered approach in the Nigeria banking sector especially with the increase in cybercrime. By proposing the use of JWT, AES-GCM, push notifications and Argon2, when implemented correctly can reduce the rate of these threats in banking systems.

CHAPTER 5:
BIBLOGRAPHY

1. Adeyeye, T. (2024, August 29). *JWT Signing Algorithms: *S256 family - What You Need to Know*. Evaluating JWT Signing Algorithms: Security, Performance, and Best Practices for Production-Grade Software Systems; CodeScroll. https://timothy.hashnode.dev/jwt-signing-algorithms-s256-family-what-you-need-to-know
2. adulau. (n.d.). *GitHub - adulau/hotp-js: A simple Javascript HOTP implementation (HMAC-Based One-Time Password Algorithm) as described in RFC 4226. The library relies on crypto-js (http://code.google.com/p/crypto-js/) for the javascript HMAC-SHA1 implementation.* GitHub. Retrieved April 6, 2025, from https://github.com/adulau/hotp-js
3. *AES GCM Stream Spec - Apache Iceberg™*. (n.d.). Apache.org. https://iceberg.apache.org/gcm-stream-spec/#cipher-block-structure
4. Agbada, A. O. (2008). Electronic Banking In Nigeria: Problems And Prospects From The Electronic Banking In Nigeria: Problems And Prospects From The Consumer's Perspective Consumer's Perspective. *CBN Bulliion*, *32*(4), 12–2008. https://dc.cbn.gov.ng/cgi/viewcontent.cgi?article=1287&context=bullion
5. Ajinkya Karode. (2023, July 19). *JWT Authentication using RS256 Algorithm in Java - Ajinkya Karode - Medium*. Medium. https://ajinkyakarode24.medium.com/jwt-authentication-using-rs256-algorithm-in-java-a3dbb9cacb78
6. Andres. (2018, October). *Recommended asymmetric algorithms for JWT?* Information

Security Stack Exchange. https://security.stackexchange.com/questions/194830/recommended-asymmetric-algorithms-for-jwt

7. Arent, B. (2024, August). *What is Time-Based One-Time Password & How it Works*. Goteleport.com. https://goteleport.com/learn/totp-time-based-one-time-password/
8. Argon2. (2024). *Argon2*. Argon2. https://www.argon2.com/
9. Auth0. (2017, March 13). *Critical Vulnerability in JSON Web Encryption*. Auth0 - Blog. https://auth0.com/blog/critical-vulnerability-in-json-web-encryption/
10. Authy. (n.d.). *Authy*. Authy. https://authy.com/what-is-2fa/
11. baeldung. (2023, May 17). *Baeldung*. Baeldung. https://www.baeldung.com/java-argon2-hashing
12. Balu, R. (n.d.). *Bangladesh Bank Cyber Heist: Incident Analysis* [THESIS]. https://repository.gatech.edu/server/api/core/bitstreams/df3d1c19-47be-4738-a855-9c049b709d52/content
13. Bejamas. (2024, November 23). *The Developer's Guide to JWT Storage*. Descope. https://www.descope.com/blog/post/developer-guide-jwt-storage
14. Bellare, M., Canetti, R., & Krawczyk, H. (1996). Message Authentication using Hash Functions| The HMAC Construction. *CrytoBytes*, *2*(1). https://cseweb.ucsd.edu/~mihir/papers/hmac-cb.pdf
15. Bernstein, C., Cobb, M., & Awati, R. (2024, February). *What Is the Advanced Encryption Standard (AES)?* SearchSecurity. https://www.techtarget.com/searchsecurity/definition/Advanced-Encryption-Standard
16. Biryukov, A., Dinu, D., & Khovratovich, D. (2015). *Argon2: the memory-hard function for password hashing and other applications*. https://www.password-

hashing.net/argon2-specs.pdf

17. Biryukov, A., Dinu, D., Khovratovich, D., & Josefsson, S. (2021, September). *RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. IETF Datatracker. https://datatracker.ietf.org/doc/rfc9106/
18. Bodewes, M. (2019, March 24). *HMAC vs ECDSA for JWT*. Cryptography Stack Exchange. https://crypto.stackexchange.com/questions/30657/hmac-vs-ecdsa-for-jwt
19. Brady, S. (2020, August 18). *JWTs: Which Signing Algorithm Should I Use?* Scott Brady. https://www.scottbrady.io/jose/jwts-which-signing-algorithm-should-i-use
20. Buchanan, B. (2023, October 16). *What Is The Fastest Signing and Verification Method for Digital Signatures*. Medium. https://billatnapier.medium.com/what-is-the-fastest-signing-and-verification-method-for-digital-signatures-0b2df0d61529
21. Bureau, B. O. (2024, November 4). *Okta Reveals Vulnerability That Allowed Login Without Password: Report*. BW Businessworld. https://www.businessworld.in/article/okta-reveals-vulnerability-that-allowed-login-without-password-report-537976
22. BusinessDay. (2024, August 19). *GTBank's cyber attack: A wake-up call for Nigerian banks amid recapitalisation efforts - Businessday NG*. Businessday NG. https://businessday.ng/companies/article/gtbanks-cyber-attack-a-wake-up-call-for-nigerian-banks-amid-recapitalisation-efforts/
23. Charity, D. T. (2024, April 3). *How to Hash Passwords with bcrypt in Node.js*. FreeCodeCamp.org. https://www.freecodecamp.org/news/how-to-hash-passwords-with-bcrypt-in-nodejs/
24. *Choose the best package - Snyk Open Source Advisor*. (2025). Snyk Advisor. https://snyk.io/advisor/

25. cilwerner. (2023, October 23). *Access token claims reference - Microsoft identity platform*. Learn.microsoft.com. https://learn.microsoft.com/en-us/entra/identity-platform/access-token-claims-reference
26. Cobb, M. (2021, November). *RSA algorithm (Rivest-Shamir-Adleman)*. TechTarget. https://www.techtarget.com/searchsecurity/definition/RSA
27. Doan, S. (2024, December 29). *Implementing TOTP Authentication in Go: A Practical Guide*. Medium. https://medium.com/@sanhdoan/implementing-totp-authentication-in-go-a-practical-guide-6fe39a349035
28. *EdDSA and Ed25519 - Practical Cryptography for Developers*. (2022). Nakov.com. https://cryptobook.nakov.com/digital-signatures/eddsa-and-ed25519
29. envoyproxy. (2021). *Probable memory leak with JWT authentication with fetched public JWKS · Issue #14026 · envoyproxy/envoy*. GitHub. https://github.com/envoyproxy/envoy/issues/14026
30. *Everybody Should 2FA*. (n.d.). Twilio. https://www.authy.com/what-is-2fa/
31. Farik, M. (2017). *A Novel Recommendation To AES Limitation*. Academia.edu. https://www.academia.edu/68937047/A_Novel_Recommendation_To_AES_Limitation
32. Flynn, C., & Chen, Z. (David). (2015). *Side Channel Power Analysis of an AES-256 Bootloader*. https://eprint.iacr.org/2014/899.pdf
33. Frederico Hakamine. (2024, November 6). *Webhook Security in the Real World*. Ngrok.com; ngrok. https://ngrok.com/blog-post/get-webhooks-secure-it-depends-a-field-guide-to-webhook-security
34. GeeksForGeeks. (2025, January 6). *RSA Algorithm in Cryptography - GeeksforGeeks*. GeeksforGeeks. https://www.geeksforgeeks.org/rsa-algorithm-cryptography/

35. go-gitea. (2021). *argon2 password hash consumes too much memory · Issue #14294 · go-gitea/gitea*. GitHub. https://github.com/go-gitea/gitea/issues/14294
36. *How to Implement Two-Factor Authentication (2FA) with TOTP in Golang*. (2024, May 6). Permify.co. https://permify.co/post/two-factor-authentication-2fa-totp-golang/
37. *How to improve JWT performance in Node.js | Nearform*. (2020). Nearform. https://www.nearform.com/insights/improve-json-web-tokens-performance-in-node-js/
38. Ihazlewood. (2022, May 19). *jwtk/jjwt*. GitHub. https://github.com/jwtk/jjwt
39. Jadhao, A., & Kumbhalkar, S. (2016). Technical Review On Secure Banking Using RSA And AES Encryptor Methodologies. *IOSR Journal of Electronics and Communication Engineering (IOSR-JECE)*, *11*(1), 1-04. https://doi.org/10.9790/2834-11140104
40. Jones, M. B., Bradley, J., & Sakimura, N. (2015, May 1). *JSON Web Token (JWT)*. IETF. https://datatracker.ietf.org/doc/rfc7519/
41. Josefsson, S., & Ilari Liusvaara. (2020, January 21). *RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)*. IETF Datatracker. https://datatracker.ietf.org/doc/rfc8032/
42. *jwe-jws*. (n.d.). Visa.com. Retrieved April 6, 2025, from https://developer.visa.com/pages/encryption_guide/jwe-jws
43. *JWT Token Security Best Practices | Curity*. (n.d.). Curity.io. https://curity.io/resources/learn/jwt-best-practices/
44. *JWT with HMAC protection*. (2024). Connect2id. https://connect2id.com/products/nimbus-jose-jwt/examples/jwt-with-hmac
45. Kahrer, J. (2022, June 20). *JWT Signatures and EdDSA*. Curity.io; Curity.

https://curity.io/resources/learn/jwt-signatures/

46. Kariyawasam, I. (2021, May 16). *Selecting the Best AES Block Cipher Mode (AES-GCM VS AES-CBC)*. Medium. https://isuruka.medium.com/selecting-the-best-aes-block-cipher-mode-aes-gcm-vs-aes-cbc-ee3ebae173c
47. kelseyk. (2023, November 15). *[New research] How tough is bcrypt to crack? And can it keep passwords safe?* Specops Software. https://specopssoft.com/blog/hashing-algorithm-cracking-bcrypt-passwords/
48. Keyless. (2024, October 3). *Understanding the Risks of One-Time Passcode (OTP) Authentication*. Keyless. https://keyless.io/blog/post/understanding-the-risks-of-one-time-passcode-otp-authentication
49. Kim, G. (2024, March 12). *Understanding Bcrypt Rounds: Balancing Security and Performance*. DEV Community. https://dev.to/mochafreddo/understanding-bcrypt-rounds-balancing-security-and-performance-5420
50. Kinde. (2024, January 18). *Guide to Bcrypt Hashing*. Kinde Guides. https://kinde.com/guides/authentication/passwords/bcrypt-hashing-guide/
51. Lazaricheva, A. (2025, February 5). *Investors, Trump and the Illuminati: What the "Nigerian prince" scams became in 2024*. Securelist.com; Kaspersky. https://securelist.com/nigerian-scams-2024/115388/
52. Lê Hoàng Tâm. (2024, September 30). *How to protect JWT Refresh Token Strategy with AES and RSA Encryption - Thnk And Grow*. Thnk and Grow. https://blog.thnkandgrow.com/how-to-protect-jwt-refresh-token-strategy-with-aes-and-rsa-encryption/
53. Lumburovska, L., Dobreva, J., Andonov, S., Trpcheska, H., & Dimitrova, V. (n.d.). *A Comparative Analysis of HOTP and TOTP Authentication Algorithms. Which one to*

choose? Retrieved April 6, 2025, from https://stumejournals.com/journals/confsec/2021/4/131.full.pdf

54. M'Raihi, D., Hoornaert, F., Naccache, D., Mihir Bellare, & Ohad Ranen. (2020). *RFC 4226: HOTP: An HMAC-Based One-Time Password Algorithm*. IETF Datatracker. https://datatracker.ietf.org/doc/rfc4226/
55. Markström, J. (2022, February 22). *Everything you need to know about One Time Passwords!* Medium. https://swjm.blog/everything-you-need-to-know-about-one-time-passwords-268387cd3fe3
56. *Microsoft Entra ID (formerly Azure Active Directory) | Microsoft Security*. (n.d.). Www.microsoft.com. https://www.microsoft.com/en-in/security/business/identity-access/microsoft-entra-id
57. Moore, D. (n.d.). *Components of JWTs Explained*. FusionAuth. https://fusionauth.io/articles/tokens/jwt-components-explained
58. Neu, P. (2020). *Choose Argon2 Parameters for Secure Password Hashing and Login*. Ory Corp; Ory Blog. https://www.ory.sh/blog/choose-recommended-argon2-parameters-password-hashing
59. *OATH-HOTP*. (n.d.). Yubico.com. Retrieved April 6, 2025, from https://docs.yubico.com/yesdk/users-manual/application-otp/hotp.html
60. Okanlawon, T. (2021, May 21). *NEFGAD drags Netflix, old generation bank to court over abuse of OTP, customer's account details*. Pmnewsnigeria.com. https://pmnewsnigeria.com/2024/05/21/nefgad-drags-netflix-old-generation-bank-to-court-over-abuse-of-otp-customers-account-details/
61. *Okta | The Identity Standard*. (n.d.). Okta. https://www.okta.com/
62. Oladapo Rilwan. (2024, April 30). *Nigeria flags spike in brute-force attacks*.

Technology Times | Latest and Breaking Nigeria Tech News; Technology Times. https://technologytimes.ng/nigeria-flags-spike-in-brute-force-attacks/

63. Oliynyk, M. (2020, March 16). *HOTP Algorithm Explained - Protectimus Solutions*. Protectimus. https://www.protectimus.com/blog/hotp-algorithm/
64. Onesafe. (2024, December 27). *Shielding Finance: The Rise of AES-256 in Quantum Era*. Onesafe.io. https://www.onesafe.io/blog/aes-256-quantum-security-banking
65. OnlineHashCrack. (n.d.). *Unlocking the Strength of Argon2: The Future of Secure Hashing | Online Hash Crack*. Https://Www.onlinehashcrack.com/How-Argon2-Algorithm-Works.php. https://www.onlinehashcrack.com/how-argon2-algorithm-works.php
66. *Ory Blog*. (2024). Ory Corp; Ory Blog. https://www.ory.sh/blog
67. PGzlan. (2023, June 30). *Understanding AES Operation Modes*. DEV Community. https://dev.to/0xog_pg/understanding-aes-operation-modes-enhancing-security-and-confidentiality-a9d
68. Presse, F. (2025, February 26). *Nigerian Banking Fraud Has Tripled: Regulator*. Barrons; Barrons. https://www.barrons.com/news/nigerian-banking-fraud-has-tripled-regulator-d06107f7
69. Proser, Z. (2025, January 5). *WorkOS*. Workos.com. https://workos.com/blog/authquake-microsofts-mfa-system-vulnerable-to-totp-brute-force-attack
70. *Recommended asymmetric algorithms for JWT?* (2023). Information Security; Stack Exchange. https://security.stackexchange.com/questions/194830/recommended-asymmetric-algorithms-for-jwt
71. RubySec. (2018, April 30). *CVE-2018-1000539 (json-jwt): Auth tag forgery*

vulnerability with AES-GCM encrypted JWT - RubySec. Rubysec.com. https://rubysec.com/advisories/CVE-2018-1000539/

72. Santhanam, A. (2024). *What is One-Time Password (OTP) and Why It's Important for Security | Infisign*. Infisign.ai. https://www.infisign.ai/blog/what-is-one-time-password-otp-and-why-its-important-for-security
73. Schneider, J. (2024, July 17). *The History of Cryptography | IBM*. Ibm.com. https://www.ibm.com/think/topics/cryptography-history
74. SecureFlag. (n.d.). *Weak Hashing Algorithm Vulnerability*. SecureFlag Security Knowledge Base. https://knowledge-base.secureflag.com/vulnerabilities/broken_cryptography/weak_hashing_algorithm_vulnerability.html
75. Security, C. (2025, January 24). *LinkedIn*. Linkedin.com. https://www.linkedin.com/pulse/hidden-vulnerabilities-totp-why-time-based-one-time-passwords-xqorf/
76. Soni, N. (2024). *https://scholarworks.calstate.edu/downloads/s4655q85w* [THESIS]. https://scholarworks.calstate.edu/downloads/s4655q85w
77. strapi. (2023). *A hardcoded BCrypt cost factor of 10 is insecure by default · Issue #10802 · strapi/strapi*. GitHub. https://github.com/strapi/strapi/issues/10802
78. The Quantum Physicist. (2018, April 22). *Why would I ever use AES-256-CBC if AES-256-GCM is more secure?* Information Security Stack Exchange. https://security.stackexchange.com/questions/184305/why-would-i-ever-use-aes-256-cbc-if-aes-256-gcm-is-more-secure
79. visa. (n.d.). *Visa*. Visa.com. https://usa.visa.com/
80. Vuuren, van. (2019, June 17). *AES Java Encryption Performance Benchmarks - Gerrit*

Jansen van Vuuren - Medium. Medium. https://medium.com/@gerritjvv/aes-java-encryption-performance-benchmarks-3c2cb19a40e9

81. Wang, S. (2019, August 22). *The performance test on the AES modes - Highgo Software Inc.* Highgo Software Inc. - Enterprise PostgreSQL Solutions. https://www.highgo.ca/2019/08/22/the-performance-test-on-the-aes-modes/
82. Wickramasinghe, S. (2024, November 26). *RSA Algorithm in Cryptography: Rivest Shamir Adleman Explained*. Splunk-Blogs. https://www.splunk.com/en_us/blog/learn/rsa-algorithm-cryptography.html
83. Yubico. (2024). *Yubico | YubiKey strong two factor authentication*. Yubico. https://www.yubico.com/
84. Zenarmor. (2021, November 16). *What is the Advanced Encryption Standard (AES)? - zenarmor.com*. Www.zenarmor.com. https://www.zenarmor.com/docs/network-security-tutorials/what-is-advanced-encryption-standard-aes
85. Zorz, Z. (2016, January 26). *Belgian bank Crelan loses €70 million to BEC scammers*. Help Net Security. https://www.helpnetsecurity.com/2016/01/26/belgian-bank-crelan-loses-e70-million-to-bec-scammers/

CHAPTER 6

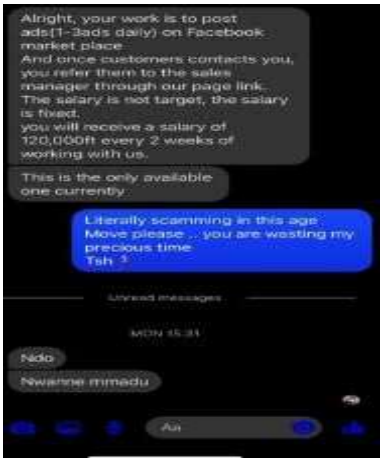
APPENDICIES

Appendix A: Evidence of attacks trying to login to my social media.

Screenshot below show the suspicious login attempts from Tashkent, Uzbekistan while I reside in Debrecen, Hungary. This shows the importance of using Argon2 for password hashing.

Appendix B: Chat messages between I as a job seeker and a potential scammer.

The image below illustrates a phishing attempt directed to me personally as a job seeker. The scammer posed as a legitimate employer who posted a role on Facebook platform. The emphasizes the need for proper awareness of fraudulent activities to prevent people from being exploited.



Appendix C: Hallelujah Challenge warning.

This image is a screenshot of Pastor Nathaniel Bassey, a Nigerian gospel minister urging people to be more vigilant for potential scammers who put on links leading to exploitation. This was stated in the last two years of the event showing the importance of the need to advance security protocols and education in Nigeria.

CHAPTER 7

DESCRIPTION OF MY WORK.

This project was chosen because of its high rate of cyber threats in Nigeria. Recently, cybercrimes have been in various forms to exploit vulnerable customers from their hard-earned money. These attacks aren't exclusive to Nigeria, but even the world at large. From my recent encounter in the Hallelujah Challenge. A yearly praise seminar organized by Nigerians; the criminals were posing as the legitimate organisers to lure the attendees of the program to pay certain amount of money. This program was meant to be free of charge, but the attackers saw a means to collect from innocent citizens. It was so severe that the organizers had to warn us to be very vigilant. The complete illustrative images will be documented in the appendix section.

This problem inspired the writing of this thesis, providing insights on advancing the security measures in financial institutions in Nigeria. The goal was to compare the existing security protocols for authentication and encryption, making us a step ahead of the attackers. The cryptanalysis was evaluated in terms of their respective security strengths, rate of speed, ease of implementation, memory space they occupy and real – world usage. A comprehensive literature review was conducted also, sighting gaps of the algorithms especially RSA and AES as they are the most common cryptographic algorithm.

Various comparisons were conducted, like Argon2 and BCrypt, JWT with different algorithms like HMAC (standard approach), RSA256, EdDSA, and AES-GCM. Proposed solutions were indicated from the gaps discovered which are: Argon2 implementing the parallelism approach and cooperation between industries with advanced machines, JWT with AES-GCM needs to go through proper checks and analysis to prevent incorrect details and TOTP with push notifications was introduced to reduce attacks like SIM swapping. Most implementations were based on previous research papers, and no direct implementation was done by me due to time constraints.

Future innovations were not left out, several solutions for the future trend was also indicated like: proper exploration of post-quantum cryptography, artificial Intelligence and machine learning can be incorporated for real time fraud detection. This journey was a personal one with lots of growth, curiosity, learning, personal and professional development that was transformed into actionable cybersecurity solutions.

CHAPTER 8
ACKNOWLEDGEMENT

I want to thank God for this opportunity to complete this project. He has been all through my journey, both highs and lows and crafted me to the person I am becoming. I would also like to thank my family, the Ogbonna's family for their efforts and support throughout my years here in University of Debrecen.

My special thanks to University of Debrecen, for the growth it offered me throughout my study, the lecturers, the events attended are appreciated. I would also like to give a special shout out to my supervisor, Dr Hannusch Carolin for going all the way for me, believing in me and her consistent efforts won't go unnoticed.